



**PapillArray Tactile Sensor
Python Library
(v1.0)**

Installation and Operation Manual

Document #: PTSPython_1.0_MAN_MAR25

March, 2025

Foreword

Information contained in this document is the property of Contactile Pty Ltd. and shall not be reproduced in whole or in part without prior written approval of Contactile Pty Ltd. The information herein is subject to change without notice and should not be construed as a commitment on Contactile Pty Ltd. This manual is periodically revised to reflect and incorporate changes made to the PapillArray Tactile Sensor Development Kit.

Contactile Pty Ltd assumes no responsibility for any errors or omissions in this document. Users' critical evaluation is welcome to assist in the preparation of future documentation.

Copyright © by Contactile Pty Ltd, Sydney, Australia. All Rights Reserved.
Published in Australia.

All trademarks belong to their respective owners.

Conditions of Sale

Contactile's conditions of sale apply to all products sold by Contactile to the Distributor under this Agreement. The conditions of sale that apply are provided on the USB flash drive shipped with the product in the folder 'LEGAL' in the root directory.

End User Licence Agreement

Contactile's end user license agreement applies to all software and algorithms included with the products sold by Contactile. The end user license agreement that applies is provided on the USB flash drive shipped with the product in the folder 'LEGAL' in the root directory.

Compliance

The devices are sold as is.

The devices are specifically designed solely for the purposes of research and development only made available on a business-to-business basis.

The devices are not for resale.

Table of Contents

1	Introduction	4
2	Safety	5
2.1	General	5
2.2	Explanation of warnings.....	5
2.3	Precautions	5
3	Getting started	6
3.1	Hardware installation	6
3.2	End user licence agreement licence.....	6
3.3	Software installation	6
3.4	Importing the library	6
4	Class and function documentation	7
4.1	PTSDKConstants class constants.....	7
4.2	PTSDKListener class public functions	8
4.3	PTSDKSensor class public functions	10
5	Writing a user application using the Python Library	12
5.1	Import files	12
5.2	Initialising PTSDKSensor and PTSDKListener objects	12
5.3	Connecting to the COM port and listening for data	12
5.4	Biasing the sensors	14
5.5	Setting the Controller sampling rate	15
5.6	Accessing sensor data.....	15
6	Log file.....	17
6.1	Overview	17
6.2	Log file location	17
6.3	Log file name.....	17
6.4	Log file format	17

1 Introduction

The PapillArray Tactile Sensor Development Kit (v2.0) is a system of (up to) two PapillArray Tactile Sensor arrays and a Controller. Each PapillArray Tactile Sensor array can measure 3D displacement, 3D force, and vibration on each sensing element, as well as global 3D force, global 3D torque, the onset of slip, and friction. The Controller supplies power for (up to) two sensors and coordinates the simultaneous data acquisition from up to two PapillArray Tactile Sensors; i.e., coordinates sampling of the 9 pillars if one sensor is connected to the Controller, 18 pillars if two sensors are connected to the Controller. The Development Kit is shipped with visualisation software, C++ and Python libraries for Windows and Linux environments and ROS and ROS2 node for developing software control algorithms using the sensor signals.

The main components of the PapillArray Tactile Sensor Development Kit (v2.0) are shown in Figure 1.1, connected to a laptop running the visualisation software.



Figure 1.1 – The PapillArray Tactile Sensor Development Kit (v2.0). Laptop not included.

This document is an installation and operation manual for the Python Library which was provided on the Contactile USB flash drive that was shipped with the Development Kit.

2 Safety

2.1 General

The customer should verify that the maximum loads and moments expected during operation fall within the sensing range of the sensor as outside this range, sensor reading accuracy is not guaranteed (refer to Document #PTS_2.0_SPEC_DEC21). Particular attention should be paid to dynamic loads caused by robot acceleration and deceleration if the sensors are mounted on robotic equipment. These forces can be many multiples of the value of static forces in high acceleration or deceleration situations.

2.2 Explanation of warnings

The warnings included here are specific to the product(s) covered by this manual. It is expected that the user heed all warnings from the manufacturers of other components used in the installation.



Danger indicates that a situation could result in potentially serious injury or damage to equipment.



Caution indicates that a situation could result in damage to the product and/or the other system components.

2.3 Precautions



DANGER: Do not attempt to disassemble the sensor. This could damage the sensor and will invalidate the calibration.



DANGER: Do not attempt to drill, tap, machine, or otherwise modify the sensor casing. This could damage the sensor and will void invalidated the calibration.



DANGER: Do not use the sensor on abrasive surfaces or surfaces with sharp points/edges. This could damage the silicone surface of the sensor.



CAUTION: Sensors may exhibit a small offset in readings when exposed to intense light sources.



CAUTION: Exceptionally strong and changing electromagnetic fields, such as those produced by magnetic resonance imaging (MRI) machines, constitute a possible source of interference with the operation of the sensor and Controller.



CAUTION: Temperature variations can cause drift in sensor readings. Some temperature compensation is performed. However, bias removal in software prior to operation is necessary, and it is recommended that biasing is performed each time the sensor is known to be unloaded.

3 Getting started

This section contains instructions for setting up and using PapillArray Tactile Sensor Python Library. It is recommended that first time users first read the preceding Safety section, then read through this section to get more familiar with the system.

3.1 Hardware installation

The Python Library is used with the PapillArray Tactile Sensor Development Kit (Beta v2.0). The Controller should be connected to the PapillArray Tactile Sensors, then the Controller should be connected via the micro USB port on the Controller to a PC running LINUX before you can use the Python Library. For more information about connecting the sensors and powering on the Controller, refer to Document #PTSDK_2.0_MAN_DEC21.

3.2 End user licence agreement licence

Contactile's end user license agreement applies to all software and algorithms included with the products sold by Contactile. The end user license agreement that applies is provided on the USB flash drive shipped with the product in the folder 'LEGAL' in the root directory.

3.3 Software installation

The Python Library is provided on the Contactile USB flash drive that was shipped with the development kit in the folders named *SOFTWARE/PythonLIN* and *SOFTWARE/PythonWIN* for Linux and Windows, respectively.

The files in the *PythonLIN* and *PythonWIN* folders are summarised in Table 3.1.

Table 3.1 – Files in *PythonLIN* and *PythonWIN* folders

Sub Folder	File Name	File Description
PythonLIN	ptsdk_cxx_pybind-1.0.0-cp310-cp310-linux_x86_64.whl	The Python wheel file for Linux
	ptsdk_pybind_example.py	The example Python code for Linux
PythonWIN	ptsdk_cxx_pybind-1.0.0-cp310-cp310-win_amd64.whl	The Python wheel file for Windows
	ptsdk_pybind_example.py	The example Python code for Windows

To install the library, in a terminal window in VSCode, navigate to the folder containing the relevant wheel file.

In Windows call: `pip install name_of_wheel_file.whl`
In Linux, call: `pip3 install name_of_wheel_file.whl`

3.4 Importing the library

In your Python code include the following: `import PTSDK_CXX_Pybind`

4 Class and function documentation

In this section, the classes and class functions of the Python Library are described.

4.1 PTSDKConstants class constants

The PTSDKConstants class contains definitions of constants that can be used in the user application. The constants are described in Table 4.1.

Table 4.1 – Constants defined in the class PTSDKConstants

Name	Value	Description
X_IND	0	The index of the X-dimension
Y_IND	1	The index of the Y-dimension
Z_IND	2	The index of the Z-dimension
NDIM	3	The number of spatial dimensions
MAX_NSENSOR	4	The maximum number of sensors connected to the Controller
MAX_NPILLAR	100	The maximum number of pillars in a sensor
CONTACT_THRESH	0.5	The minimum normal (Z) force for a pillar to be in contact
INELIGIBLE	-2	Pillar slip state: not in contact at slip detection start
CONTACT_AT_START	1	Pillar slip state: in contact from slip detection start
LOST_CONTACT	-1	Pillar slip state: has lost contact
TLOADING	2	Pillar slip state: is loaded tangentially
SLIPPED	3	Pillar slip state: has slipped
NOFRICTIONEST	-1	Value representing no friction estimate
SAMP_RATE_100	100	Constant representing 100 Hz sampling rate
SAMP_RATE_250	250	Constant representing 250 Hz sampling rate
SAMP_RATE_500	500	Constant representing 500 Hz sampling rate
SAMP_RATE_1000	1000	Constant representing 1000 Hz sampling rate

4.2 PTSDKListener class public functions

The PTSDKListener is the class which interacts with the Controller that is in turn hosting up to two connected PapillArray Tactile Sensors. This class describes an object that connects with the Controller via a serial connection emulated on the computer's USB port, and reads and processes the data streaming through the serial connection. This class also logs the data to a log file – See Section 6 Log file. The public member functions of the PTSDKListener class are described below.

`__init__(self, isLog: bool) -> None`

Description: Constructor.

Parameters: self
 isLog A flag indicating whether to log data to CSV file.

`def addSensor(self, pSensor: PTSDKSensor) -> None`

Description: Adds a sensor object to the PTSDKListener.

Parameters: self
 pSensor A pointer to the sensor object.

`def connect (port: str, rate: int, parity: int, byteSize: str) -> int`

Description: Connects to the COM port.

Used in conjunction with the readNextSample and disconnect functions.

Parameters: self
 port The COM port name.
 rate The rate of the connection.
 parity The parity of the connection.
 byteSize The byte size for the connection.

Returns: 0 if successfully connected, error code if unsuccessful.

`def connectAndStartListening(self, port: str, rate: int, parity: int, byteSize: str, isFlush: bool) -> int`

Description: Connects to the COM port and starts listening for data (starts the listening thread), processes the data and logs the data to a log file.

Used in conjunction with the stopListeningAndDisconnect function.

Parameters: self
 port The COM port name.
 rate The rate of the connection.
 parity The parity of the connection.
 byteSize The byte size for the connection.
 isFlush A flag indicating whether to flush the hardware input buffer if it contains too many bytes.

Returns: 0 if successfully connected, error code if unsuccessful.

`def disconnect(self) -> None`

Description: Disconnects from the COM port.

Used in conjunction with the connect and readNextSample functions.

`def getTargetGripForce(self) -> float`

Description: Connects to the COM port and starts listening for data (starts the listening thread), processes the data and logs the data to a log file.

Used in conjunction with the stopListeningAndDisconnect function.

Returns: The target grip force

def readNextSample(self) -> bool

Description: Reads and parses the next sample from the COM port, and stores the sample in the associated PTSDKSensor objects.

Used in conjunction with the connect and disconnect functions.

Returns: True if successfully read a sample, false if unsuccessful.

def sendBiasRequest(self) -> bool

Description: Sends a bias request to the Controller. A bias should be performed after connecting to the serial port and starting to stream data with the sensor unloaded. A bias should be performed each time the sensor is known to be unloaded. A bias operation can take up to 2 s. Ensure that the sensor remains unloaded throughout this time.

Returns: True if successfully sent the request, false if unsuccessful.

def setSamplingRate(self, *samplingRate: int*) -> bool

Description: Sets the sampling rate on the Controller.

Parameters: self
samplingRate The sampling rate for the Controller: SAMP_RATE_100, SAMP_RATE_250, SAMP_RATE_500 or SAMP_RATE_1000

Returns: True if successfully sent the request, false if unsuccessful.

def startSlipDetection(self) -> bool

Description: Starts the slip detection algorithms on the Controller. This should be called after a number of pillars of the sensor are already in contact, before tangential loading of the sensor.

Returns: True if successfully sent the request, false if unsuccessful.

def stopListeningAndDisconnect(self) -> None

Description: Stops listening for data from the COM port (and kills the listening thread), stops logging data to the log file and disconnects from the COM port.

def stopSlipDetection(self) -> bool

Description: Stops and resets the slip detection algorithms on the Controller.

Returns: True if successfully sent the request, false if unsuccessful.

4.3 PTSDKSensor class public functions

The PTSDKSensor is a class that describes a PapillArray Tactile Sensor (v2.0). This is the main class for accessing the current sensor measurements in a user-defined program.

`__init__(self)`

Description: Constructor - Initialises pillars

`def getAllDisplacements(self) -> list`

Description: Gets the current X, Y and Z displacements of all pillars of this sensor.

Returns: List[NDIM][NPILLAR]. The X, Y and Z displacements (mm) of all pillars.

`def getAllForces(self) -> list`

Description: Gets the current X, Y and Z forces of all pillars of this sensor.

Returns: List[NDIM][NPILLAR]. The X, Y and Z forces (N) of all pillars.

`def getAllSlipStatus(self) -> tuple`

Description: Gets the slip state of all pillars of the sensor.

Returns:

<code>isSlipDetectionActive</code>	True if slip detection is active, false otherwise.
<code>isRefPillarLoaded</code>	True if the tangential force on the reference pillar has exceeded the threshold, false otherwise
<code>contactStates</code>	List[NPILLAR]. For each pillar, true if the pillar normal force exceeds the threshold for contact, false otherwise.
<code>slipStates</code>	List[NPILLAR]. The slip states of all pillars: INELIGIBLE if the pillar was not in contact when slip detection started, CONTACT_AT_START if the pillar was in contact when slip detection started, LOST_CONTACT if the pillar lost contact after slip detection started, TLOADING if the pillar is being loaded tangentially, of SLIPPED if the pillar has slipped.

`def getFrictionEstimate(self) -> float`

Description: Gets the current friction estimate from this sensor.

Returns: The current friction estimate from this sensor; or NOFRICTIONESTIMATE if there is no friction estimate.

`def getGlobalForce(self) -> list`

Description: Gets the global X,Y,Z force acting on the sensor.

Parameters: List[NDIM]. The global X, Y and Z force (N).

`def getGlobalTorque(self) -> list`

Description: Gets the global X,Y,Z torque acting on the sensor.

Parameters: List[NDIM]. The global X, Y and Z torque (Nmm). The torque reference point is the current tip position of the centre pillar (P4).

`def getNPillar(self) -> int`

Description: Gets the number of pillars in this sensor.

Returns: The number of pillars in this sensor.

`def getPillarDisplacements(self, pillarInd: int) -> list`

Description: Gets the current X, Y and Z displacement of a pillar.

Parameters: pillarInd The index of the pillar.
Returns: List[NDIM]. The X, Y and Z displacement (mm) of the pillar,
 or [-1000,-1000,-1000] if the pillar index is invalid.

def getPillarForces(self, pillarInd: int) -> list

Description: Gets the current X, Y and Z force on the pillar tip.
Parameters: pillarInd The index of the pillar.
Returns: List[NDIM]. The X, Y and Z force (N) of the pillar,
 or [-1000,-1000,-1000] if the pillar index is invalid.

def getPillarForceAbs(self, pillarInd: int) -> float

Description: Gets the current absolute force (force magnitude) on the pillar tip.
Parameters: pillarInd The index of the pillar.
Returns: The current absolute force (N) on the pillar tip,
 or -1000 if the pillar index is invalid.

def getPillarForceN(self, pillarInd: int) -> float

Description: Gets the current normal (Z) force on the pillar tip.
Parameters: pillarInd The index of the pillar.
Returns: The current normal (Z) force (N) on the pillar tip,
 or -1000 if the pillar index is invalid.

def getPillarForceT(self, pillarInd: int) -> float

Description: Gets the current tangential (XY) force.
Parameters: pillarInd The index of the pillar.
Returns: The current tangential force (N) on the pillar tip,
 or -1000 if the pillar index is invalid.

def getTargetGripForce(self) -> float

Description: Gets the current target grip force from this sensor.
Returns: The current target grip force from this sensor.

def getTimestamp_us(self) -> int

Description: Gets the timestamp of the current sample of a pillar in μ s.
Returns: The timestamp of the current sample of a pillar in μ s.

def isSensorInContact(self) -> bool

Description: Gets whether the sensor is in contact.
Returns: True if at least one pillar is in contact; false otherwise.

5 Writing a user application using the Python Library

This section contains code snippets to explain each step required to write a user application that uses the Python Library to monitor two PapillArray Tactile Sensors. The full example can be found in the `ptsdk_pybind_example.py` file in the relevant *PythonXXX* folder (where XXX is WIN or LIN for Windows and Linux, respectively).

5.1 Import files

These examples require the imports listed in Example 5.1.

Example 5.1 – Imports for the example user application

```
import PTSDK_CXX_Pybind
```

5.2 Initialising PTSDKSensor and PTSDKListener objects

To initialise a PTSDKListener object, first, the PTSDKSensor objects must be initialised. The following information is required to initialise the PTSDKSensor objects. An example of initialising two PTSDKSensor objects then initialising the PTSDKListener object is shown in Example 5.2.

Example 5.2 – Initialising two PTSDKSensor objects and a PTSDKListener object

```
isLogging = True

# Initialise sensors
sen0 = PTSDK_CXX_Pybind.PTSDKSensor()
sen1 = PTSDK_CXX_Pybind.PTSDKSensor()

# Initialise listener and add sensors
listener = PTSDK_CXX_Pybind.PTSDKListener(isLogging)
listener.addSensor(sen0)
listener.addSensor(sen1)
```

5.3 Connecting to the COM port and listening for data

After initialising the PTSDKListener, a serial connection must be established. To connect to the COM port, the name of the COM port assigned to the connected Controller must be known. Once the PTSDKListener has established a connection with the COM port of the Controller, the Controller will begin transmitting data through the serial connection.

There are two methods by which a user defined program can retrieve data from the Controller:

1. Single thread
2. Multi-threaded

Note: There should be a COM port associated with the Controller (to power the Controller, the micro-USB should be connected between the Controller and the PC).

Note: In Linux, the COM port usually appears as `/dev/ttyACM0`. Ensure that the user has permissions to read/write to this COM port (by adding the user to the dialout group).

5.3.1 COM port configuration parameters

The COM port configuration parameters are first required. An example of initialising the COM port configuration parameters is shown in Example 5.3.

Example 5.3 – Connecting the PTSDKListener object to the COM port and listen for data in a single thread

```
port = "\\\\.\\COM3"    # The name of the COM port to connect with
                        # For Windows, the additional '\\' characters are required
                        # For Linux, this could be "/dev/ttyACM0"
rate = 115200           # The rate of the serial connection
parity = 0              # 0=PARITY_NONE, 1=PARITY_ODD, 2=PARITY_EVEN
byteSize = '\u0008'     # The number of bits in a byte, as a single Unicode character
```

5.3.2 Single thread

The structure of a user defined application using a single thread to retrieve sensor data from the Controller is shown in Example 5.4.

Example 5.4 – Connecting to the COM port and reading data in a single thread

```
# Single thread example
isFlush = True
pillarInd = 0
s0p0forces = [0,0,0]

# Connect to the COM port
res = listener.connect(port, rate, parity, byteSize)
if res == 0:
    print('main(): Successfully connected to the COM port')
else:
    print('main(): FAILED to connect to the COM port and start to listen')
    exit()

# Read the next sample
for i in range(0,1000):
    res = listener.readNextSample(True)
    if not res:
        print("main(): FAILED to read the next sample")
        break
    # User application code - do something with the data
    s0p0forces = sen0.getPillarForces(pillarInd)
    if i%100 == 0:
        for dimInd in range(0,PTSDK_CXX_Pybind.PTSDKConstants.NDIM):
            print('S0P0: F' + str(dimInd) + ': ' + str(s0p0forces[dimInd]))

# Disconnect from the COM port
listener.disconnect()
```

5.3.3 Multi-threaded

The PTSDKListener object can launch a thread which listens for and processes the incoming data packets. An example of how to connect to the COM port and start listening for data using a new thread is shown in Example 5.5.

Example 5.5 – Connecting to the COM port and listening for data in a multi-threaded application

```
# Multi-threaded example
pillarInd = 0
s0p0disp = [0,0,0]

# Connect to COM port and start listening
isFlush = True
res = listener.connectAndStartListening(port, rate, parity, byteSize, isFlush)
if res == 0:
    print('main(): Successfully connected COM port and starting to listen')
else:
    print('main(): FAILED to connect to COM port and start to listen')
    exit()

# User application code - do something with the data
for i in range(0,10):
    force = sen0.getGlobalForce()
    s0p0disp = sen0.getPillarDisplacements(pillarInd)
    for dimInd in range(0,PTSDK_CXX_Pybind.PTSDKConstants.NDIM):
        print('S0: global F' + str(dimInd) + ' = ' + str(force[dimInd]))
        print('S0P0: D' + str(dimInd) + ': ' + str(s0p0disp[dimInd]))

    time.sleep(1)

# Stop listening and disconnect from COM port
listener.stopListeningAndDisconnect()
```

5.4 Biasing the sensors

Biasing refers to removing any offset in the pillar readings when the pillars are unloaded. It is recommended that the user performs a bias each time the sensors are known to be unloaded. Ensure that the sensor has been unloaded for at least one second before performing a bias to ensure that the bias calculation does not include hysteresis effects. A bias operation can take up to 2 s. Ensure that the sensor remains unloaded throughout this time. An example of how to perform a bias is shown in Example 5.6.

Example 5.6 – Biasing all pillars on all sensors

```
# Send a BIAS request
res = listener.sendBiasRequest()
if res:
    print('main(): Successfully sent a BIAS request')
else:
    print('main(): FAILED to send a BIAS request')
    exit()
```

5.5 Setting the Controller sampling rate

By default, upon powering up, the Controller will default to a sample rate of 1000 Hz. The Controller sampling rate can be changed to 100, 250, 500 or 1000 Hz. An example of how to change the Controller sampling rate to 500 Hz is shown in Example 5.7.

Example 5.7 – Setting the Controller sampling rate

```
# Setting the controller sampling rate
res = listener.setSamplingRate(PTSDK_CXX_Pybind.PTSDKConstants.SAMP_RATE_500)
if res:
    print('main(): Successfully set the sampling rate')
else:
    print('main(): FAILED to set the sampling rate')
    exit()
```

5.6 Accessing sensor data

Once the PTSDKListener object is listening for and processing data and the sensors have been biased, the user application can access the incoming sensor data. Examples of how to access different types of data are shown in Example 5.8, Example 5.9 and Example 5.10.

Example 5.8 – Accessing data from a whole sensor

```
# Accessing data from a whole sensor
for i in range(0,10):
    force = sen0.getGlobalForce() # Get XYZ Global Force from SEN0
    for dimInd in range(0,PTSDK_CXX_Pybind.PTSDKConstants.NDIM):
        print('S0: global F' + str(dimInd) + ' = ' + str(force[dimInd]))
    time.sleep(1)
for i in range(0,10):
    s1disp = sen1.getAllDisplacements() # Get XYZ displacements of all pillars of SEN1
    for pInd in range(0,sen1.getNPillar()):
        print('S1P' + str(pInd) + ': D0 = ' + str(s1disp[0][pInd]) + ', D1 = ' +
              str(s1disp[1][pInd]) + ', D2 = ' + str(s1disp[2][pInd]))
    time.sleep(1)
```

Example 5.9 – Accessing data from a single pillar

```
# Accessing data from a single pillar
pillarInd = 3
for i in range(0,10):
    force = sen0.getPillarDisplacements(pillarInd) # Get P3 force from SEN0
    for dimInd in range(0,PTSDK_CXX_Pybind.PTSDKConstants.NDIM):
        print('S0P3: F' + str(dimInd) + ': ' + str(force[dimInd]))
    time.sleep(1)
```

Example 5.10 – Performing slip detection and estimating friction

```
# Performing slip detection and estimating friction
res = listener.startSlipDetection() # Start slip detection
if res:
    print('main(): Successfully started slip detection')
else:
    print('main(): FAILED to start slip detection')
    exit()
isSlipDetectionActive, isRefPillarLoaded, contactStates, slipStates =
    sen0.getAllSlipStatus() # Get slip states of all pillars in SEN0
for pillarInd in range(0, sen0.getNPillar()):
    print('S0_P' + str(pillarInd) + ':')
    match slipStates[pillarInd]:
        case PTSDK_CXX_Pybind.PTSDKConstants.INELIGIBLE:
            print('\tWas not in contact at slip detection start.')
        case PTSDK_CXX_Pybind.PTSDKConstants.CONTACT_AT_START:
            print('\tIn contact rom slip detection start.')
        case PTSDK_CXX_Pybind.PTSDKConstants.LOST_CONTACT:
            print('\tLost contact.')
        case PTSDK_CXX_Pybind.PTSDKConstants.TLOADING:
            print('\tIs being tangentially loaded.')
        case PTSDK_CXX_Pybind.PTSDKConstants.SLIPPED:
            print('\tSlipped.')
friction = sen0.getFrictionEstimate() # Get the current friction estimate of SEN0
print('S0: friction estimate = ' + str(friction))
res = listener.startSlipDetection() # Stop slip detection
if res:
    print('main(): Successfully stopped slip detection')
else:
    print('main(): FAILED to stop slip detection')
    exit()
```


6 Log file

6.1 Overview

If the PTSDKListener object was initialised with the isLogging flag being true, the function connectAndStartListening (in a multi-threaded application) and the PTSDK function readNextSample (in a single thread application) also generate a log file of the sensor data.

6.2 Log file location

The log file that is generated is stored in the *Logs* subfolder in the same location as the user-defined application which uses the Python Library.

6.3 Log file name

The name of the log file that is generated is LOG_YYYY_MM_DD_hh_mm_ss.csv where:

- YYYY is the four digit year,
- MM is the two digit month,
- DD is the two digit day,
- hh is the two digit hour,
- mm is the two digit minute and
- ss is the two digit second,

from the system clock at the time that the log file was created.

6.4 Log file format

The log file is saved as comma-separated values (CSV) in ASCII text format. The order of the values and a description is shown in Table 6.1

Table 6.1 – Data in log file

Data Order	Data Name	Data Description	
1	T_us	Timestamp in μ s	Timestamp
2	S0_P0_DX	Sensor 0, pillar 0, X-axis displacement (mm)	Sensor 0, pillar 0, displacements
3	S0_P0_DY	Sensor 0, pillar 0, Y-axis displacement (mm)	
4	S0_P0_DZ	Sensor 0, pillar 0, Z-axis displacement (mm)	
5	S0_P0_FX	Sensor 0, pillar 0, X-axis force (N)	Sensor 0, pillar 0, forces
6	S0_P0_FY	Sensor 0, pillar 0, Y-axis force (N)	
7	S0_P0_FZ	Sensor 0, pillar 0, Z-axis force (N)	
8	S0_P1_DX	Sensor 0, pillar 1, X-axis displacement (mm)	Sensor 0, pillar 1, displacements
9	S0_P1_DY	Sensor 0, pillar 1, Y-axis displacement (mm)	
10	S0_P1_DZ	Sensor 0, pillar 1, Z-axis displacement (mm)	
11	S0_P1_FX	Sensor 0, pillar 1, X-axis force (N)	Sensor 0, pillar 1, forces
12	S0_P1_FY	Sensor 0, pillar 1, Y-axis force (N)	
13	S0_P1_FZ	Sensor 0, pillar 1, Z-axis force (N)	
⋮			

Data Order	Data Name	Data Description	
50	S0_P8_DX	Sensor 0, pillar 8, X-axis displacement (mm)	Sensor 0, pillar 8, displacements
51	S0_P8_DY	Sensor 0, pillar 8, Y-axis displacement (mm)	
52	S0_P8_DZ	Sensor 0, pillar 8, Z-axis displacement (mm)	
53	S0_P8_FX	Sensor 0, pillar 8, X-axis force (N)	Sensor 0, pillar 8, forces
54	S0_P8_FY	Sensor 0, pillar 8, Y-axis force (N)	
55	S0_P8_FZ	Sensor 0, pillar 8, Z-axis force (N)	
56	S0_GG_FX	Sensor 0, global X-axis force (N)	Sensor 0, global force
57	S0_GG_FY	Sensor 0, global Y-axis force (N)	
58	S0_GG_FZ	Sensor 0, global Z-axis force (N)	
59	S0_GG_TX	Sensor 0, global X-axis torque (Nmm) ^	Sensor 0, global torque ^
60	S0_GG_TY	Sensor 0, global Y-axis torque (Nmm) ^	
61	S0_GG_TZ	Sensor 0, global Z-axis torque (Nmm) ^	
62	S0_isSDActive	Sensor 0, is slip detection activated	Sensor 0 slip detection and friction estimate
63	S0_isRefLoad	Sensor 0, is the reference pillar tangentially loaded	
64	S0_P0_isInContact	Sensor 0, pillar 0, is in contact	
65	S0_P0_slipState	Sensor 0, pillar 0, slip state	
66	S0_P1_isInContact	Sensor 0, pillar 1, is in contact	
67	S0_P1_slipState	Sensor 0, pillar 1, slip state	
		⋮	
80	S0_P8_isInContact	Sensor 0, pillar 8, is in contact	
81	S0_P8_slipState	Sensor 0, pillar 8, slip state	
82	S0_FrictionEst	Sensor 0, friction estimate	
83	S1_P0_DX	Sensor 1, pillar 0, X-axis displacement (mm)	Sensor 1, pillar 0, displacements
84	S1_P0_DY	Sensor 1, pillar 0, Y-axis displacement (mm)	
85	S1_P0_DZ	Sensor 1, pillar 0, Z-axis displacement (mm)	
86	S1_P0_FX	Sensor 1, pillar 0, X-axis force (N)	Sensor 1, pillar 0, forces
87	S1_P0_FY	Sensor 1, pillar 0, Y-axis force (N)	
88	S1_P0_FZ	Sensor 1, pillar 0, Z-axis force (N)	
89	S1_P1_DX	Sensor 1, pillar 1, X-axis displacement (mm)	Sensor 1, pillar 1, displacements
90	S1_P1_DY	Sensor 1, pillar 1, Y-axis displacement (mm)	
91	S1_P1_DZ	Sensor 1, pillar 1, Z-axis displacement (mm)	
92	S1_P1_FX	Sensor 1, pillar 1, X-axis force (N)	Sensor 1, pillar 1, forces
93	S1_P1_FY	Sensor 1, pillar 1, Y-axis force (N)	
94	S1_P1_FZ	Sensor 1, pillar 1, Z-axis force (N)	
		...	
131	S1_P8_DX	Sensor 1, pillar 8, X-axis displacement (mm)	Sensor 1, pillar 8, displacements
132	S1_P8_DY	Sensor 1, pillar 8, Y-axis displacement (mm)	
133	S1_P8_DZ	Sensor 1, pillar 8, Z-axis displacement (mm)	

Data Order	Data Name	Data Description	
134	S1_P8_FX	Sensor 1, pillar 8, X-axis force (N)	Sensor 1, pillar 8, forces
135	S1_P8_FY	Sensor 1, pillar 8, Y-axis force (N)	
136	S1_P8_FZ	Sensor 1, pillar 8, Z-axis force (N)	
137	S1_GG_FX	Sensor 1, global X-axis force (N)	Sensor 1, global force
138	S1_GG_FY	Sensor 1, global y-axis force (N)	
139	S1_GG_FZ	Sensor 1, global Z-axis force (N)	
140	S1_GG_TX	Sensor 1, global X-axis torque (Nmm) ^	Sensor 1, global torque ^
141	S1_GG_TY	Sensor 1, global Y-axis torque (Nmm) ^	
142	S1_GG_TZ	Sensor 1, global Z-axis torque (Nmm) ^	
143	S1_isSDActive	Sensor 1, is slip detection activated	Sensor 1 slip detection and friction estimate
144	S1_isRefLoad	Sensor 1, is the reference pillar tangentially loaded	
145	S1_P0_isInContact	Sensor 1, pillar 0, is in contact	
146	S1_P0_slipState	Sensor 1, pillar 0, slip state	
147	S1_P1_isInContact	Sensor 1, pillar 1, is in contact	
148	S1_P1_slipState	Sensor 1, pillar 1, slip state	
		⋮	
161	S1_P8_isInContact	Sensor 1, pillar 8, is in contact	
162	S1_P8_slipState	Sensor 1, pillar 8, slip state	
163	S1_FrictionEst	Sensor 1, friction estimate	

^ The torque reference point is the current tip position of the centre pillar (P4).